

Desenvolvimento do Sistema de Processamento Visual (SPV)



Grupo 8

Ronaldo Ávila de Arruda Junior 11202231908

Leonardo Garcia Dos Santos 11202230441

Santo André – SP
2026

1. Requisitos de Ambiente

O código foi desenvolvido em **Python** e utiliza bibliotecas de processamento de imagem e matrizes.

Bibliotecas Necessárias

Você precisará instalar as seguintes dependências via terminal ou prompt de comando:

Bash

```
pip install opencv-python numpy
```

- **opencv-python (cv2):** Responsável pela captura de vídeo, manipulação de cores, detecção de contornos e interface gráfica.
- **numpy:** Utilizada para cálculos matemáticos e manipulação das matrizes das imagens.
- **os e datetime:** Bibliotecas nativas do Python (não precisam de instalação) usadas para criar pastas e dar nome aos arquivos de log.

Ambiente Jupyter Notebook / Lab

Embora o código funcione no Jupyter, ele abre janelas externas do OpenCV (cv2.imshow). No Jupyter, às vezes o kernel pode travar ao fechar as janelas se não for feito corretamente.

Dica: Se as janelas travarem, reinicie o Kernel do Jupyter.

2. Estrutura do Projeto

Ao executar o código, o sistema criará automaticamente:

- Uma pasta chamada /falhas: Onde serão salvas as fotos de todas as peças identificadas como "REPROVADO".
- **Interface Interativa:** Uma janela de vídeo com botões clicáveis para ajuste de parâmetros.

3. Guia de Execução Passo a Passo

Passo 1: Preparação

1. Certifique-se de ter uma webcam conectada ao computador.
2. Coloque o script em uma pasta onde você tenha permissão de escrita (para a criação da pasta de falhas).

Passo 2: Execução

1. Execute a célula do Jupyter ou o arquivo .py.

2. Uma janela chamada "**Inspecao de Qualidade**" abrirá mostrando o feed da sua câmera.

Passo 3: Calibração (Interface do Usuário)

O código possui um painel interativo no canto superior esquerdo. Você pode clicar com o **botão esquerdo do mouse** diretamente nos botões [+] e [-] na tela para ajustar:

- **Sensibilidade BlackHat:** Melhora a detecção de trincas escuras em superfícies claras.
- **Tol. Superfície:** Define quantos pixels de "falha" são aceitáveis antes de reprovar a peça.
- **Área Mín. Quebra:** Ajusta o tamanho mínimo de um defeito na borda para que ele seja considerado uma "quebra".

Passo 4: Operação de Inspeção

1. **Posicionamento:** Coloque a peça (preferencialmente vermelha) dentro do retângulo azul central.
2. **Análise:** Aperte a tecla **ESPAÇO**. O sistema irá "congelar" o frame e processar a imagem.
3. **Resultado:** * Se aparecer **APROVADO** (Verde): A peça está dentro dos parâmetros.
 - Se aparecer **REPROVADO** (Vermelho): O sistema identificará falhas e salvará automaticamente uma imagem na pasta falhas/.
4. **Debug:** Uma segunda janela ("Painel de Debug CV") abrirá mostrando as etapas do processamento (máscaras de cor, filtros e detecção de trincas).
5. **Retomar:** Aperte **ESPAÇO** novamente para voltar ao modo de visualização ao vivo.

Passo 5: Finalização

- Para fechar o programa completamente, pressione a tecla **ESC**.

4. Resumo das Teclas de Atalho

Tecla	Ação
ESPAÇO	Alterna entre modo de visualização e modo de análise (congelar/processar).
ESC	Fecha o programa e encerra todas as janelas.

Clique Mouse	Ajusta os parâmetros de calibração nos botões + e -.
---------------------	--

5. Lógica Técnica (O que o código faz?)

1. **Segmentação:** Ele isola a cor vermelha usando o espaço de cor **HSV**, que é mais robusto a variações de iluminação.
2. **Análise de Borda:** Usa o algoritmo de **Convexity Defects** (Defeitos de Convexidade). Se a borda da peça "entrar" muito para dentro, ele identifica como uma quebra.
3. **Análise de Superfície:** Aplica um filtro **BlackHat** (que destaca elementos escuros menores que o kernel em fundos claros) para encontrar rachaduras ou riscos.
4. **Decisão:** Se a área de trincas for maior que a tolerância OU houver quebras na borda, a peça é reprovada.

Referência: Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media.

Gonzalez, R. C., & Woods, R. E. (2017): *Digital Image Processing*. Pearson.